Privilege OpenAPI MCP on Kubernetes: Working Patterns and Remaining Gap

Originally prepared: 2026-10-01 · **Updated:** 2026-10-03

Purpose: Explain the OpenAPI-to-MCP patterns recorded in this repository and compare them with the supplied engineering ask, “Enable Kubernetes-Native Deployment for Generated OpenAPI MCP Apps.”

Summary

The repository records two different ways to make OpenAPI-described APIs available through Privilege:

1. **Native Privilege OpenAPI app (`banking-rest`).** Privilege receives a REST API endpoint and an OpenAPI spec URL. A statically configured Kubernetes sidecar makes both reachable from the gateway Pod. Privilege turns the API operations into MCP tools.
2. **Self-hosted OpenAPI adapter (`banking-openapi`).** The Ping `mcp/openapi` image runs as a configured sidecar. It reads an API and spec from another sidecar and exposes an MCP endpoint on the gateway Pod. The operator notes prescribe registering that endpoint as an MCP Server. The current BFF preset calls the same door an “OpenAPI MCP Agentic App”; because the live console record is not checked in, the repository does not settle that UI-type discrepancy.

Both patterns depend on workloads configured ahead of time through Helm and a manually configured Privilege app. Neither watches each generated descriptor and creates a Kubernetes Deployment and Service for it. The Petstore workflow in the supplied PDF therefore remains a separate automation gap. This report distinguishes the repository's `ai-demo-cmuir` / `ping-devops-curtismuir` setup from the PDF's `ping-devops-tarunmadiraju` environment.

Pattern	Where OpenAPI becomes MCP	Kubernetes workload	Evidence and limit
`banking-rest`	Privilege's native **Add OpenAPI MCP** app	REST API sidecar on port `8082`	Operational notes record the app config and a live REST-path check. That check proves the API path, not a current Privilege tool invocation.
`banking-openapi`	Self-hosted `mcp/openapi` sidecar	REST sidecar on `8082` plus MCP adapter on `8080`	Sidecar values and BFF preset record the config and discovered tools. The checked-in notes disagree on the app's exact console type.

What failed and what changed

The original `openapi2` app used the Privilege-managed `mcp/openapi` image. Investigation measured `POST /mcp initialize` returning `200` and producing tools when the image was run with valid configuration, while `GET /mcp` returned `405`; the image had no `mcp-shim`. The repository's investigation attributes the `openapi2` failure to the managed image packaging and runtime path. That is an observed result for this setup, not a claim about every Privilege installation. See CURRENT-CONFIGURATION.md.

The supplied PDF describes a related but different problem: Privilege wrote a runtime descriptor for Petstore, but no Kubernetes workload appeared, and the gateway service account could not create Pods or Deployments. The repository's approach keeps Kubernetes deployment under Helm/operator control. It does not grant the gateway process permission to launch containers at runtime.

Pattern 1: Privilege's native OpenAPI app (`banking-rest`)

The operational notes describe a native **Add OpenAPI MCP** application named `banking-rest`. This app takes a REST API endpoint and an OpenAPI spec URL; Privilege creates the MCP tools. The Kubernetes sidecar remains an ordinary REST API and does not implement MCP JSON-RPC.

The recorded configuration is:

Setting	Value
Privilege app type/name	**Add OpenAPI MCP** / `banking-rest`
REST API endpoint	`http://localhost:8082`
OpenAPI spec URL	`http://localhost:8082/openapi/banking-rest.json`
Mesh cluster	`ai-demo-cmuir`
Sidecar	`mcp-banking-rest`, port `8082`, in the `agentless-mcpgw` Pod
API key	Held by the REST sidecar; Privilege does not receive it

The sidecar proxies banking routes to `mcp-resource-server` and serves the OpenAPI document. The notes record a live request from inside the sidecar to `/banking` returning seeded accounts, which verifies the sidecar-to-Service route and API-key authentication. The source notes do not provide a captured request proving a current Privilege-generated tool call, so this report keeps those two verification claims separate. See AGENTLESS-CONFIGURATION.md and server.js.

This is the clearest native OpenAPI pattern in the repo: Privilege handles OpenAPI-to-tool conversion, and the cluster supplies a statically deployed REST endpoint and spec. The gateway Pod's shared network namespace lets the app use `localhost:8082`; the workload is not created from the generated app descriptor.

Pattern 2: Run Ping's OpenAPI adapter as a sidecar (`banking-openapi`)

The other path runs Ping's `mcp/openapi` adapter in the gateway Pod and gives it a separate REST API sidecar. The checked-in Helm values configure:

```
Adapter image:       public.ecr.aws/n2z2g8w6/mcp/openapi:latest
Adapter listener:   8080
REST API endpoint:   http://localhost:8082
OpenAPI spec:        http://localhost:8082/openapi/simple-users.json
Tool mode:           both
Readiness check:     TCP port 8080
```

The adapter serves MCP on `http://localhost:8080/mcp`. Its current spec is the self-contained Simple User API, used as a controlled adapter/discovery probe; it is not the banking API spec from the `banking-rest` configuration. The TCP readiness probe is used because the adapter returns `405` to `GET /mcp` even though its MCP initialization is a POST. See sidecars.values.yaml and the REST sidecar routes.

The operator instructions in `CURRENT-CONFIGURATION.md` specify a manually configured **MCP Server** app named `banking-openapi`, with URL `http://localhost:8080/mcp`. The BFF's current Privilege door preset calls it an "OpenAPI MCP Agentic App" and records the tools `call_api`, `get_endpoint`, `listUsers`, and `list_endpoints`. These are different descriptions in checked-in records. Since the live console object is external to the repository, this report treats the MCP Server tile as the documented setup instructions and does not claim an independently verified current console type.

This adapter-sidecar setup is separate from `banking-rest`: here the sidecar itself converts the OpenAPI spec into MCP, and the gateway reaches its `/mcp` endpoint. The supported static sidecar approach avoids requiring a Kubernetes API client or create permissions inside the gateway process.

Kubernetes deployment mechanics

The vendored Helm chart template adds configured `extraContainers` to the gateway Pod. Kubernetes schedules those containers with the Pod, and they share its network namespace. That makes the `localhost:8080` and `localhost:8082` endpoints available to the gateway without a separate Service between the containers. See deployment.yaml.

The local chart change must be preserved when the chart is re-vendored. Also, Helm replaces the `extraContainers` list as a whole; an upgrade must pass the complete list of sidecars or omitted containers can disappear. The repo keeps the sidecar definitions in sidecars.values.yaml.

What this proves—and what remains

- The `banking-rest` record shows how a native Privilege OpenAPI app can reach a REST API and its spec inside the gateway Pod. Its notes verify the REST route and API-key path.
- The self-hosted adapter values show how to run Ping's OpenAPI MCP binary as a fixed sidecar, use a TCP probe for its POST-oriented endpoint, and connect it to an API/spec sidecar.
- The BFF preset records that `banking-openapi` exposes generated and discovery tools. The repository has conflicting descriptions of its console app type and no checked-in live console export.
- The current sidecar spec URL is `simple-users.json`. It does not prove that the current banking proxy target matches the separate cross-namespace target in older operational notes.
- The later `banking-mcp` app is a direct MCP integration. It does not exercise OpenAPI conversion.

The engineering ask calls for more: when Privilege produces an arbitrary OpenAPI app descriptor, a controller or deployment service would need to create/update a Kubernetes Deployment and Service, supply the API/spec configuration, publish a reachable backend URL, and reconcile deletion or descriptor changes. This repo contains static Helm sidecars and manual app registration; it does not contain that descriptor watcher or per-app lifecycle automation. The static patterns are useful for known APIs at deploy time, but they do not close the PDF's generated-app workflow.

Source files

- AGENTLESS-CONFIGURATION.md — native `banking-rest` OpenAPI app setup and recorded live REST-path check.
- CURRENT-CONFIGURATION.md — the original `openapi2` investigation and self-hosted adapter instructions.
- sidecars.values.yaml — adapter and REST sidecar definitions, image, environment, ports, and probes.
- deployment.yaml — Helm hook that injects `extraContainers`.
- privilegeMcpClient.js — current `banking-openapi` door preset and tool description.
- demo_mcp_banking_rest/server.js and server tests — REST/OpenAPI probe and banking proxy.
- banking-rest.openapi.json and resource server — banking REST schema and implementation.